Initialization

int[] numbers = {10, 20, 30, 40};

One-Dimensional Arrays

A **one-dimensional array** stores elements in a single row

Example:

```
public class OneDArray {
   public static void main(String[] args) {
      int[] numbers = {10, 20, 30, 40, 50};
      System.out.println(numbers[0]);
      System.out.println(numbers[3]);
   }
}
```

Array Traversal

Traversal means accessing every element in the array.

Using for loop

```
int[] arr = {1,2,3,4,5};

for(int i = 0; i < arr.length; i++) {
   System.out.println(arr[i]);
}
```

Using enhanced for loop

```
for(int value : arr) {
   System.out.println(value);
}
```

Two-Dimensional Arrays

A **two-dimensional array** stores elements in **rows and columns** (matrix form).

Declaration

```
int[][] matrix;
```

Creation

```
matrix = new int[3][3];
```

Example

```
int[][] matrix = {
  {1,2,3},
  {4,5,6},
  {7,8,9}
};
for(int i = 0; i < 3; i++) {
  for(int j = 0; j < 3; j++) {
    System.out.print(matrix[i][j] + " ");
  }
  System.out.println();
}
```

Output:

```
1 2 3
4 5 6
7 8 9
```

Multidimensional Arrays

Java supports **arrays with more than two dimensions**.

Example:

```
int[][][] cube = new int[2][3][4];
```

Accessing element:

```
cube[1][2][3] = 50;
```

Such arrays are used in **scientific computing, graphics, and simulations**.

gged Arrays

A **jagged array** is an array where **rows have different column sizes**.

Unlike regular matrices, each row can store a different number of elements.

Example:

```
int[][] jagged = new int[3][];

jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
```

Visualization

```
Row0 → [x x]
Row1 → [x x x x]
Row2 → [x x x]
```

Example program:

```
for(int i = 0; i < jagged.length; i++) {

   for(int j = 0; j < jagged[i].length; j++) {

      System.out.print(jagged[i][j] + " ");

   }

   System.out.println();
}
```

Array Memory Representation

When an array is created:

1. Reference variable is stored in **stack memory**
2. Actual array object is stored in **heap memory**

Example:

```
int[] arr = {10,20,30};
```

Memory representation

```
Stack Memory      Heap Memory
-------------     ----------------
arr  -----------> [10,20,30]
```

Each element has an index:

```
Index:  0  1  2
Value:  10 20 30
```

Array Operations

Common operations performed on arrays include:

- Traversal
- Searching
- Sorting
- Insertion
- Deletion

Searching Algorithms

Searching means **finding the position of an element in an array**.

Two common algorithms are:

1. Linear Search
2. Binary Search

Linear Search

In **linear search**, each element is checked sequentially.

Algorithm

1. Start from first element
2. Compare with target value
3. If match found → return index
4. Otherwise continue until end

Example

```
int[] arr = {10,20,30,40,50};
int key = 30;
for(int i = 0; i < arr.length; i++) {
  if(arr[i] == key) {
    System.out.println("Found at index " + i);
    break;
  }
}
```

Binary Search

Binary search works on **sorted arrays**.

Algorithm

1. Find middle element
2. Compare with target
3. If equal → found
4. If smaller → search left
5. If larger → search right

Example:

```
int[] arr = {10,20,30,40,50};
int key = 30;
int low = 0;
int high = arr.length - 1;
while(low <= high) {
  int mid = (low + high) / 2;
  if(arr[mid] == key) {
    System.out.println("Found at index " + mid);
    break;
  }
  else if(arr[mid] < key)
    low = mid + 1;
```

```
  else
    high = mid - 1;
}
}
```

Time Complexity

O(log n)

Sorting Algorithms

Sorting means **arranging array elements in ascending or descending order**.

Common sorting algorithms:

- Bubble Sort
- Selection Sort
- Insertion Sort

Bubble Sort

In bubble sort, adjacent elements are compared and swapped if they are in the wrong order.

Example:

```
int[] arr = {5,3,8,4,2};

for(int i = 0; i < arr.length-1; i++) {

  for(int j = 0; j < arr.length-i-1; j++) {

    if(arr[j] > arr[j+1]) {

      int temp = arr[j];
      arr[j] = arr[j+1];
      arr[j+1] = temp;

    }

  }

}
```

Output

2 3 4 5 8

Time Complexity

$O(n^2)$

Strings in Java

A **String** in Java represents a sequence of characters. It is one of the most widely used classes in Java programs for storing and manipulating text data such as names, messages, file paths, and user inputs.

In Java, strings are implemented using the **String** class from the java.lang package. Since java.lang is automatically imported, the String class can be used directly in any Java program.

Example:

String message = "Hello Java";

Here:

- String → class type
- message → reference variable
- "Hello Java" → string literal

Strings are **objects**, not primitive data types.

Creating Strings in Java

There are two primary ways to create strings.

1. Using String Literal

String s = "Java";

This string is stored in the **String Constant Pool (SCP)**.

---

2. Using new Keyword

String s = new String("Java");

This creates a **new object in heap memory**, even if the same value already exists in the String pool.

String Class

The **String** class provides many methods for string manipulation.

Some important operations include:

- Concatenation
- Comparison
- Substring extraction
- Searching characters
- Converting case

Example:

String name = "Java Programming";

Important String Methods

Method	Description
length()	Returns string length
charAt()	Returns character at index
substring()	Extracts substring
equals()	Compares string content
equalsIgnoreCase()	Compares ignoring case
toUpperCase()	Converts to uppercase
toLowerCase()	Converts to lowercase
contains()	Checks if substring exists
replace()	Replaces characters

Example

```
String str = "Java";

System.out.println(str.length());
System.out.println(str.toUpperCase());
System.out.println(str.charAt(1));
```

Output

4
JAVA
a

String Comparison

Strings can be compared in two ways.

1. Using == Operator

== compares **memory references**, not actual content.

Example:

```
String s1 = "Java";
String s2 = "Java";

System.out.println(s1 == s2);
```

Output

true

Both references point to the **same object in the String pool**.

---

2. Using equals() Method

equals() compares **actual content**.

Example:

```
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1.equals(s2));
```

Output

true

Even though objects are different, contents are the same.

Immutable Strings

Strings in Java are **immutable**, meaning once a string object is created, its value **cannot be changed**.

Example:

```
String s = "Java";
s.concat(" Programming");
```

The original string remains unchanged.

Why Java Strings are Immutable:

1. Security
2. Thread safety
3. Efficient memory usage
4. String pool optimization

Immutability is enforced by the **String** class design.

String Constant Pool (SCP)

The **String Constant Pool** is a special memory region in the heap where Java stores string literals.

Purpose of SCP:

- Avoid duplicate string objects
- Improve memory efficiency
- Improve performance

Example:

```
String s1 = "Java";
String s2 = "Java";
```

Both references point to the **same object**.

Memory representation:

```
Stack Memory      Heap (String Pool)
------------      --------------------
s1  ----------->  "Java"
s2  ----------->  "Java"
```

Internal Working of String Constant Pool

When a string literal is created:

1. JVM checks the **String Pool**
2. If the string already exists → reference is reused
3. If not → new string object is created

Example:

```
String s1 = "Java";
String s2 = new String("Java");
```

Memory representation:

```
Stack          Heap
-----          ------------------
s1 ----------->   "Java" (SCP)

s2 ----------->   new String object
```

Thus:

s1 == s2 → false

But:

s1.equals(s2) → true

StringBuilder Class

The **StringBuilder** class is used to create **mutable strings**.

Mutable means the string content **can be modified without creating a new object**.

Example:

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");

System.out.println(sb);
```

Output

Java Programming

Advantages:

- Faster than String
- No new objects created for modification
- Efficient for large string manipulations

StringBuffer Class

The **StringBuffer** class is similar to StringBuilder but **thread-safe**.

Example:

```
StringBuffer sb = new StringBuffer("Java");

sb.append(" Programming");

System.out.println(sb);
```

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread safety	Yes	No	Yes
Performance	Slow for modification	Fast	Slower than StringBuilder
Memory	Uses String Pool	Heap	Heap

When to Use Each

Use **String** when:

- Data is constant
- String pool optimization is needed

Use **StringBuilder** when:

- Frequent modifications occur
- Single-threaded environment

Use **StringBuffer** when:

- Thread-safe string manipulation is required

Strings are heavily optimized in Java because they are used very frequently in programs. The **String** class includes several internal optimizations in the **Java Virtual Machine** to improve memory efficiency and performance.

This section explains **internal working, memory representation, and advanced string manipulation techniques**.

Internal Representation of String

Internally, a String object stores characters inside a **character array**.

Simplified representation:

```
final class String {
   private final char[] value;
}
```

Explanation:

- final → prevents modification after creation
- char[] value → stores characters of the string
- immutability ensures thread safety and memory optimization

Example:

String s = "JAVA"

Internal storage:

['J','A','V','A']

Because the character array is **final**, the contents cannot be modified.

String Interning

Interning is a process where the JVM stores only **one copy of identical string literals** in the **String Constant Pool**.

Example:

```
String s1 = "Java";
String s2 = "Java";
```

Memory representation:

```
Stack      Heap (String Pool)
-----      -------------------
s1  ----->   "Java"
s2  ----->   "Java"
```

Both references point to the **same object**.

---

intern() Method

The intern() method forces a string into the **String Constant Pool**.

Example:

```
String s1 = new String("Java");
String s2 = s1.intern();

System.out.println(s1 == s2);
```

Explanation:

- s1 → heap object
- s2 → reference from string pool

String Concatenation

Concatenation means **combining two or more strings**.

There are several ways to concatenate strings.

---

1 Using + Operator

```
String s1 = "Java";
String s2 = "Programming";

String result = s1 + s2;
```

Internally the compiler converts this into **StringBuilder operations**.

Equivalent code:

```
new StringBuilder().append(s1).append(s2).toString();
```

---

2 Using concat() Method

```
String s1 = "Java";
String s2 = "Programming";

String result = s1.concat(s2);
```

Compile-Time vs Runtime Concatenation

Compile-Time Concatenation

```
String s = "Java" + "Programming";
```

The compiler automatically converts it into:

"JavaProgramming"

Stored directly in the **String Pool**.

Runtime Concatenation

```
String s1 = "Java";
String s2 = "Programming";

String s3 = s1 + s2;
```

Here the result is created **during program execution**, usually using **StringBuilder**.

String Memory Behavior Example

Example:

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
```

Memory diagram:

```
Stack        Heap
-----        ----------------------
s1 ------->   "Java"  (String Pool)

s2 ------->   "Java"  (same object)

s3 ------->   new String("Java")  (Heap)
```

Results:

```
s1 == s2 → true
s1 == s3 → false
s1.equals(s3) → true
```

Common String Manipulation Methods

substring()

Extracts part of a string.

```
String text = "Programming";

System.out.println(text.substring(0,4));
```

Output:

Prog

---

indexOf()

Finds position of character.

```
String text = "Java Programming";

System.out.println(text.indexOf("P"));
```

---

split()

Splits string into parts.

```
String data = "apple,banana,orange";

String[] fruits = data.split(",");
```

Result:

["apple", "banana", "orange"]

---

trim()

Removes spaces.

```
String text = "  Java  ";

System.out.println(text.trim());
```

String Performance Considerations

Because **String** objects are immutable, frequent modifications create many temporary objects.

Example (inefficient):

```
String s = "";

for(int i=0;i<1000;i++){
   s = s + i;
}
```

This creates **1000 new string objects**.

Better Solution

Use **StringBuilder**.

```
StringBuilder sb = new StringBuilder();

for(int i=0;i<1000;i++){
   sb.append(i);
}

String result = sb.toString();
```

This improves **performance and memory usage**.

StringBuilder Internal Working

The **StringBuilder** uses a **dynamic character array** internally.

Example capacity growth:

Initial capacity → 16
If exceeded → (capacity × 2) + 2

So capacity increases like:

16 → 34 → 70 → ...

StringBuffer Internal Working

The **StringBuffer** is similar to StringBuilder but includes **synchronization** for thread safety.

Example:

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
```

Because of synchronization:

- Thread safe
- Slightly slower

Real World Uses of Strings

Strings are used in:

- File paths
- User input

- Network communication
- Database queries
- JSON / XML processing
- Logging systems

Because of heavy usage, the JVM provides **String Pool optimization**.

Best Practices for Using Strings

1. Use **string literals** instead of new String() whenever possible.
2. Use **StringBuilder** for heavy concatenation.
3. Use **equals() instead of ==** for comparison.
4. Avoid unnecessary string creation.
5. Use **intern() carefully** when memory optimization is required.

Packages in Java

A **package** in Java is a namespace that groups related classes and interfaces together. Packages help organize large applications, avoid naming conflicts, and provide access protection.

Packages are managed by the **Java Virtual Machine** and are part of Java's modular structure.

Example:

package com.example.myapp;

This statement places the class inside the com.example.myapp package.

Why Packages Are Used

Packages provide several advantages:

1. **Code organization** – groups related classes.
2. **Avoids naming conflicts** – two classes with the same name can exist in different packages.
3. **Access control** – used with access modifiers.
4. **Code reuse and maintainability**.

Example structure:

```
com
└── company
    └── project
        ├── Main.java
        ├── User.java
        └── Database.java
```

Types of Packages

Java supports two types of packages:

1. Built-in packages
2. User-defined packages

Built-in Packages

Java provides many predefined packages containing commonly used classes. These are called **built-in packages**.

Examples include:

Package	Purpose
java.lang	Fundamental classes
java.util	Utility classes
java.io	Input and output
java.net	Networking
java.sql	Database connectivity

The **java.lang** package is automatically imported.

Example classes in java.lang:

- **String**
- **System**
- **Math**

Example usage:

```
System.out.println("Hello Java");
```

User-Defined Packages

Developers can create their own packages to organize application code.

Creating a Package

package com.company.project;

Example program:

```
package com.example;

public class Test {
  public void display() {
    System.out.println("User defined package example");
  }
}
```

Directory structure:

```
com
└── example
    └── Test.java
```

Compilation:

```
javac -d . Test.java
```

The package Keyword

The package keyword is used to define the package to which a class belongs.

Rules:

- Must be the **first statement in a Java file**
- Only **one package declaration per file**

Example:

```
package com.mycompany.project;

public class Demo {
  public static void main(String[] args) {
    System.out.println("Package example");
  }
}
```

The import Keyword

The import keyword allows classes from other packages to be used.

Syntax:

```
import packageName.className;
```

Example:

```
import java.util.Scanner;
```

Here the class **Scanner** from the **java.util** package is imported.

Example program:

```
import java.util.Scanner;

public class InputExample {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.println("Enter number:");
      int num = sc.nextInt();

      System.out.println("Number is " + num);

   }

}
```

Types of Import

1 Single Class Import

```
import java.util.Scanner;
```

---

2 Wildcard Import

```
import java.util.*;
```

This imports **all classes from the package**.

Static Import

Static import allows accessing static members directly without class name.

Example:

```
import static java.lang.Math.*;

public class Test {
  public static void main(String[] args) {
    System.out.println(sqrt(16));
  }
}
```

Methods in Java

A **method** in Java is a block of code that performs a specific task. Methods help improve **code reusability, readability, and modularity**.

When a method is executed, the instructions are interpreted and executed by the **Java Virtual Machine**.

Example:

```
public void greet() {
  System.out.println("Hello Java");
}
```

Here:

- public → access modifier
- void → return type
- greet() → method name
- { } → method body

Method Declaration

A method declaration defines the structure of a method.

Syntax

```
accessModifier returnType methodName(parameters) {
  // method body
}
```

Example:

```
public int add(int a, int b) {
   return a + b;
}
```

Explanation:

Part	Description
public	Access modifier
int	Return type
add	Method name
int a, int b	Parameters

Method Calling (Method Invocation)

A method must be **called** to execute.

Example:

```
public class Test {

   public static void main(String[] args) {

      Test obj = new Test();
      obj.display();

   }

   void display() {
      System.out.println("Method called");
   }

}
```

When display() is called:

1. JVM creates a **new stack frame**
2. Executes method instructions
3. Removes stack frame after execution

Where Methods Are Stored in JVM

Understanding **JVM memory architecture** helps understand method behavior.

Memory areas involved:

JVM Memory Area	Purpose
Heap	Objects stored
Stack	Method execution
Method Area	Class metadata
Program Counter	Current instruction

Method Storage

Method information is stored in the **Method Area** of the **Java Virtual Machine**.

The Method Area contains:

- method bytecode
- class metadata
- static variables
- constant pool

Example memory representation:

```
Method Area
----------------------
Class metadata
Method bytecode
Static variables
Constant pool
```

Method Execution in Stack

When a method is called:

1. JVM creates a **stack frame**
2. Parameters and local variables are stored
3. Method executes
4. Stack frame is removed after completion

Example:

```
public static void main(String[] args) {
  add(5,10);
```

```
}

static int add(int a,int b){
   return a + b;
}
```

Stack behavior:

```
Stack Memory
-----------------
main()
add(5,10)
```

After completion:

```
Stack Memory
-------------
main()
```

Parameters and Arguments

Parameters

Variables declared in method definition.

```
void display(int number)
```

Here number is a **parameter**.

Arguments

Values passed during method call.

```
display(10);
```

Here 10 is an **argument**.

Types of Parameters

Java supports several parameter types:

1. Value parameters
2. Reference parameters

Pass by Value

Java always uses **pass by value**.

Example:

```
void change(int num){
   num = 50;
}

int x = 10;
change(x);

System.out.println(x);
```

Output:

10

Reason:

A **copy of value** is passed.

Object Reference Passing

Example:

```
class Test {
   int value = 10;
}

void modify(Test obj){
   obj.value = 50;
}
```

Here object data can change because **reference copy points to same object**.

Return Types

Methods can return a value using the return statement.

Example:

```
int square(int num){
   return num * num;
}
```

Calling method:

```
int result = square(5);
```

Void Return Type

If a method does not return a value:

```
void display(){
  System.out.println("Hello");
}
```

Method Overloading

Method overloading means defining multiple methods with the **same name but different parameters**.

Example:

```
class Calculator {

  int add(int a,int b){
    return a+b;
  }

  int add(int a,int b,int c){
    return a+b+c;
  }

}
```

Rules:

- Same method name
- Different parameter list
- Return type alone cannot differentiate methods

Compile-Time Polymorphism

Method overloading is resolved at **compile time**.

The Java compiler determines which method to call based on parameters.

Varargs (Variable Arguments)

Varargs allow passing **multiple arguments to a method**.

Syntax:

```
datatype... variableName
```

Example:

```
void sum(int... numbers){

   int total = 0;

   for(int n : numbers){
      total += n;
   }

   System.out.println(total);
}
```

Method calls:

```
sum(10,20);
sum(10,20,30,40);
```

Internally, varargs are implemented as **arrays**.

Equivalent form:

```
void sum(int[] numbers)
```

Recursion

Recursion occurs when a method **calls itself**.

Example: factorial

```
int factorial(int n){

   if(n == 1)
      return 1;

   return n * factorial(n-1);

}
```

Example execution:

```
factorial(5)
5 * factorial(4)
5 * 4 * factorial(3)
...
```

Recursion Stack Behavior

Each recursive call creates a **new stack frame**.

Example stack:

factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)

After reaching base condition, calls return in reverse order.

Stack Overflow in Recursion

If recursion does not have a **base condition**, stack memory may overflow.

Example:

```
void recursive(){
   recursive();
}
```

Result:

StackOverflowError

Static Methods vs Instance Methods

Feature	Static Method	Instance Method
Belongs to	Class	Object
Called using	Class name	Object
Memory	Method area	Method area

Example static method:

```
static void display(){
   System.out.println("Static method");
}
```

Call:

```
Test.display();
```

Best Practices for Methods

1. Use meaningful method names
2. Keep methods small
3. Avoid deep recursion
4. Use varargs carefully
5. Reduce side effects

Concept	Description
Method	Block of reusable code
Method Declaration	Defines method structure
Method Calling	Executes method
Parameters	Variables in method
Arguments	Values passed
Overloading	Same method with different parameters
Varargs	Variable arguments
Recursion	Method calling itself

Object-Oriented Programming (OOP) Basics in Java

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects rather than functions and logic**.

Java is a **purely object-oriented language** (except primitive types). OOP concepts allow developers to build **modular, reusable, and scalable systems**.

All objects in Java are managed by the **Java Virtual Machine** and stored in **heap memory** during program execution.

Main OOP building blocks:

1. Class
2. Object
3. Fields (variables)
4. Methods (behavior)
5. Constructors

Class in Java

A **class** is a blueprint or template used to create objects.

It defines:

- properties (fields)
- behavior (methods)

Example:

```
class Student {
  int id;
  String name;

  void display() {
    System.out.println(id + " " + name);
  }
}
```

Here:

- Student → class
- id, name → fields
- display() → method

Where Class Is Stored in JVM

When a class is compiled:

Student.java → Student.class

The .class file contains **bytecode**.

During execution, the **Java Virtual Machine** loads the class into the **Method Area**.

Method Area stores:

Class metadata
Method bytecode
Static variables
Constant pool
Field information

Object in Java

An **object** is a runtime instance of a class.

Objects contain:

- state (data)
- behavior (methods)

Example:

Student s1 = new Student();

Here:

Part	Meaning
Student	class
s1	reference variable
new	object creation keyword
Student()	constructor

Object Creation Process (JVM Internals)

When this line executes:

Student s1 = new Student();

Steps inside the **Java Virtual Machine**:

1. JVM checks if class is loaded
2. Memory allocated in **Heap**
3. Fields initialized to default values
4. Constructor executed
5. Reference stored in **Stack**

Memory representation:

```
Stack Memory      Heap Memory
-------------                  ----------------
s1  ----------->   Student Object
              id = 0
              name = null
```

Fields and Methods

Fields

Fields represent **object properties**.

Example:

```
class Car {
   String brand;
   int speed;
}
```

Fields are stored **inside objects in heap memory**.

Methods

Methods represent **behavior** of objects.

Example:

```
void accelerate() {
   speed = speed + 10;
}
```

Methods are stored in the **Method Area**.

Constructors in Java

A **constructor** is a special method used to initialize objects.

Characteristics:

- Same name as class
- No return type
- Automatically called when object is created

Example:

```
class Student {

  Student() {
    System.out.println("Constructor called");
  }

}
```

Object creation:

```
Student s = new Student();
```

Default Constructor

If a class does not define any constructor, the compiler provides a **default constructor**.

Example:

```
class Test {
}
```

Compiler internally creates:

```
Test() {
}
```

Purpose:

- initialize object
- assign default values

Parameterized Constructor

Constructors can accept parameters to initialize object data.

Example:

```
class Student {
  int id;
  String name;
  Student(int i, String n) {
    id = i;
    name = n;
  }

}
```

Object creation:

```
Student s1 = new Student(101, "John");
```

Constructor Overloading

Multiple constructors with different parameters.

Example:

```
class Student {

  int id;
  String name;

  Student() {
    id = 0;
    name = "Unknown";
  }

  Student(int i, String n) {
    id = i;
    name = n;
  }

}
```

The compiler decides which constructor to call based on arguments.

Constructor Chaining

Constructor chaining means **calling one constructor from another constructor**.

Two keywords used:

this()
super()

Example:

```
class Student {
  int id;
  String name;
  Student() {
    this(0,"Unknown");
  }
```

```
  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }

}
```

Here this() calls another constructor.

Rule:

this() must be first statement

---

this Keyword

The **this keyword** refers to the **current object**.

Example:

```
class Student {
  int id;
  Student(int id){
    this.id = id;
  }

}
```

Without this, the parameter would hide the field.

Uses of this:

1. Refer current object
2. Call another constructor
3. Pass current object
4. Return current object

---

static Keyword

The **static keyword** means **belonging to the class rather than an object**.

Example:

```
class Counter {
  static int count = 0;
  Counter() {
    count++;
```

```
}
}
```

All objects share the same static variable.

Memory behavior:

```
Method Area
-------------
static variables

Heap
-------------
object data
```

Static Method Example

```
class MathUtil {
   static int square(int x){
      return x*x;
   }
}
```

Calling method:

```
MathUtil.square(5);
```

No object required.

---

final Keyword

The **final keyword** restricts modification.

It can be used with:

Usage	Meaning
final variable	constant
final method	cannot override
final class	cannot inherit

Example:

```
final int MAX = 100;
```

Example final class:

```
final class Utility {
}
```

A final class **cannot be extended**.

Access Modifiers

Access modifiers control **visibility and accessibility** of classes, methods, and variables.

Java provides four access modifiers:

1. private
2. default
3. protected
4. public

Private Access Modifier

Accessible only **within the same class**.

Example:

```
class Test {

   private int number;

}
```

Usage:

- data hiding
- encapsulation

Default Access Modifier

When no modifier is specified.

Accessible **within same package**.

Example:

```
class Test {
   int value;
}
```

Protected Access Modifier

Accessible:

- within same package
- subclasses in different packages

Example:

```
protected int data;
```

Used mainly in **inheritance**.

Public Access Modifier

Accessible **from anywhere in the program**.

Example:

```
public class Test {
}
```

Used for APIs and library classes.

Access Modifier Comparison

Modifier	Same Class	Same Package	Subclass	Other Packages
private	YES	NO	NO	NO
default	YES	YES	NO	NO
protected	YES	YES	YES	NO
public	YES	YES	YES	YES

Why Access Modifiers Are Important

They help achieve:

1. **Encapsulation**
2. **Security**
3. **Controlled access**
4. **Maintainability**

Example:

Private fields + Public getters/setters

Real World OOP Example

Example class:

```
class BankAccount {
  private double balance;
  BankAccount(double balance){
    this.balance = balance;
  }
  void deposit(double amount){
    balance += amount;
  }
  double getBalance(){
    return balance;
  }
}
```

Object creation:

```
BankAccount acc = new BankAccount(5000);
```

Concept	Description
Class	Blueprint for objects
Object	Instance of class
Field	Object property
Method	Object behavior
Constructor	Initializes objects
this	Current object reference
static	Class-level member
final	Prevent modification
Access Modifiers	Control visibility

Records in Java

A **Record** is a special type of class introduced in **Java 14** as a preview feature and officially finalized in **Java 16**.

Records provide a **compact syntax for declaring immutable data objects**.

They are mainly used to represent **data carriers (DTOs)** without writing boilerplate code like:

- getters
- constructors
- equals()
- hashCode()
- toString()

Records are implemented as a special type of class handled by the **Java Virtual Machine**.

Why Records Were Introduced

Before records, creating simple data classes required a lot of boilerplate code.

Example without records:

```
class Person {

  private final String name;
  private final int age;

  public Person(String name, int age) {
    this.name = name;
    this.age = age;
  }

  public String name() {
    return name;
  }

  public int age() {
    return age;
  }
}
```

This requires:

- fields
- constructor

- getters
- equals()
- hashCode()
- toString()

Records eliminate most of this.

Declaring a Record

Syntax:

```
record RecordName(type field1, type field2) {
}
```

Example:

```
record Person(String name, int age) {
}
```

This single line automatically generates:

- private final fields
- constructor
- accessor methods
- equals()
- hashCode()
- toString()

What JVM Generates Internally

When the compiler sees:

```
record Person(String name, int age) {}
```

The compiler generates something similar to:

```
final class Person extends java.lang.Record {

  private final String name;
  private final int age;

  public Person(String name, int age) {
    this.name = name;
    this.age = age;
```

```
    }

    public String name() {
        return name;
    }

    public int age() {
        return age;
    }

    public boolean equals(Object o) { ... }

    public int hashCode() { ... }

    public String toString() { ... }
}
```

Important points:

- Records **implicitly extend java.lang.Record**
- They are **final classes**
- Fields are **final**

Creating Record Objects

Example:

```
record Person(String name, int age) {}

public class Main {

    public static void main(String[] args) {

        Person p = new Person("John", 25);

        System.out.println(p.name());
        System.out.println(p.age());

    }

}
```

Output:

```
John
25
```

Record Accessor Methods

Record components automatically create accessor methods.

Example:

```
Person p = new Person("Alice", 30);
```

Accessing values:

```
p.name();
p.age();
```

Note:

Records **do not use getter naming convention** (getName()).

They use:

```
componentName()
```

Record Immutability

All record fields are **implicitly final**.

Example:

```
record Student(String name, int marks) {}
```

Attempting modification:

```
Student s = new Student("Tom", 90);
s.name = "Sam";   // ERROR
```

This ensures **immutability**.

Benefits:

- thread safety
- predictable behavior
- safe data sharing

Compact Constructors

Records allow custom constructors called **compact constructors**.

Example:

```
record Person(String name, int age) {

  public Person {
    if(age < 0)
      throw new IllegalArgumentException("Age cannot be negative");
  }

}
```

Here:

- validation logic is added
- field assignment handled automatically

Custom Constructors

You can define constructors explicitly.

Example:

```
record Person(String name, int age) {

  public Person(String name) {
    this(name, 18);
  }

}
```

Adding Methods in Records

Records can contain methods like normal classes.

Example:

```
record Rectangle(int length, int width) {

  int area() {
    return length * width;
  }

}
```

Usage:

```
Rectangle r = new Rectangle(10,5);
System.out.println(r.area());
```

Output:

```
50
```

Restrictions of Records

Records have some limitations.

Restriction	Reason
Cannot extend another class	Already extends Record
Fields are final	Immutability
Cannot declare instance fields	Components define state
Cannot be abstract	Must be concrete

Records vs Classes

Feature	Class	Record
Boilerplate code	High	Minimal
Mutability	Mutable/Immutable	Immutable
Fields	Flexible	Final
Inheritance	Supported	Not supported
Primary purpose	General objects	Data carriers

Records vs Lombok

Many developers previously used **Project Lombok** to generate boilerplate code.

Example with Lombok:

```
@Data
@AllArgsConstructor
class Person {
   private String name;
```

```
   private int age;
}
```

Records remove the need for such libraries.

Records in JVM Memory

Records behave like normal objects.

Memory representation:

```
Stack              Heap
------            ----------------
p  --------->       Person object
                 name = "John"
                 age  = 25
```

Stored in:

- **Heap memory**
- Methods stored in **Method Area**

Managed by the **Java Virtual Machine**.

Use Cases of Records

Records are ideal for:

- Data Transfer Objects (DTO)
- API responses
- Database query results
- Configuration objects
- Immutable models

Example:

```
record UserResponse(int id, String name, String email) {}
```

Records with Collections

Example:

```
record Book(String title, double price) {}

List<Book> books = List.of(
```

```
    new Book("Java", 500),
    new Book("Python", 400)
);
```

Object-Oriented Programming Principles in Java

Object-Oriented Programming (OOP) organizes software design around **objects, classes, and interactions between them**.
Java implements OOP concepts using classes and objects managed by the **Java Virtual Machine**.

The four fundamental OOP principles are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

These principles help build **modular, reusable, and maintainable software systems**.

---

1. Encapsulation

Concept

Encapsulation means **binding data and methods together into a single unit (class)** and **restricting direct access to data**.

It is also called **data hiding**.

Example:

```
class BankAccount {

    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }

}
```

Here:

- balance → hidden using private
- Access controlled through methods.

Why Encapsulation is Important

Encapsulation provides:

- **Security**
- **Data protection**
- **Controlled modification**
- **Better maintainability**

Example problem without encapsulation:

account.balance = -1000;

Encapsulation prevents such illegal modifications.

Internal Working

In compiled bytecode:

- fields are marked with **access flags**
- the **JVM verifies access rules**

Example bytecode flag:

ACC_PRIVATE
ACC_PUBLIC

During execution the **Java Virtual Machine** enforces these access restrictions.

---

2. Inheritance

Concept

Inheritance allows one class to **acquire properties and methods of another class**.

It enables:

- code reuse
- hierarchical relationships

Example:

```
class Animal {
  void eat() {
    System.out.println("Animal eats");
  }

}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }

}
```

Here:

Dog → child class
Animal → parent class

Types of Inheritance in Java

Java supports **three main inheritance types with classes**.

---

1. Single Inheritance

One child inherits from one parent.

A → B

Example:

```
class A {
  void display(){
    System.out.println("Class A");
  }
}

class B extends A {
}
```

---

2. Multilevel Inheritance

Inheritance chain across multiple levels.

A → B → C

Example:

```
class A {
  void show(){
    System.out.println("A");
  }
}
class B extends A {
}
class C extends B {
}
```

---

3. Hierarchical Inheritance

Multiple classes inherit from one parent.

```
    A
  /   \
  B    C
```

Example:

```
class Animal {}

class Dog extends Animal {}

class Cat extends Animal {}
```

Why Java Does Not Support Multiple Inheritance (Classes)

Java avoids **diamond problem**.

Example problem:

```
   A
  / \
 B   C
  \ /
   D
```

If both B and C define the same method, JVM cannot decide which one D should inherit.

Instead Java uses **interfaces** to support multiple inheritance safely.

---

3. Polymorphism

Concept

Polymorphism means **many forms**.

In Java it allows **one interface to represent multiple implementations**.

Two types:

1. Compile-time polymorphism
2. Runtime polymorphism

Method Overloading (Compile-Time Polymorphism)

Multiple methods with **same name but different parameters**.

Example:

```
class Calculator {

  int add(int a, int b){
    return a + b;
  }

  double add(double a, double b){
    return a + b;
  }

}
```

Compiler decides which method to call based on **method signature**.

Method Signature

method name + parameter types

Return type **does not participate** in method overloading.

Method Overriding (Runtime Polymorphism)

Occurs when a **child class provides its own implementation of a parent method**.

Example:

```
class Animal {

  void sound(){
    System.out.println("Animal sound");
  }

}

class Dog extends Animal {

  @Override
  void sound(){
    System.out.println("Dog barking");
  }

}
```

Runtime Method Dispatch

Method selection occurs at runtime.

Example:

```
Animal a = new Dog();
a.sound();
```

Output:

Dog barking

Why?

The **actual object type** determines the method execution.

This mechanism is implemented by the **Java Virtual Machine** using **dynamic method dispatch**.

---

4. Abstraction

Concept

Abstraction hides **implementation details** and exposes only **essential features**.

Example:

Driving a car:

You use steering wheel
You do not see engine internals

In Java abstraction is achieved using:

1. Abstract classes
2. Interfaces

Abstract Class

An abstract class **cannot be instantiated** and may contain **abstract methods**.

Example:

```
abstract class Shape {

   abstract double area();

}
```

Child class:

```
class Circle extends Shape {

   double radius;

   Circle(double r){
      radius = r;
   }

   double area(){
      return 3.14 * radius * radius;
   }

}
```

Abstract Class Features

Feature	Description
May contain abstract methods	Yes
May contain concrete methods	Yes
Constructors allowed	Yes
Fields allowed	Yes

Interface

An interface defines **a contract that classes must implement**.

Example:

```
interface Vehicle {

   void start();

}
```

Implementation:

```
class Car implements Vehicle {

   public void start(){
      System.out.println("Car started");
   }

}
```

Interface Characteristics

Feature	Interface
Methods	abstract by default
Variables	public static final
Multiple inheritance	allowed
Constructors	not allowed

Default Methods in Interfaces

Introduced in **Java 8**.

Allows interfaces to have **method implementations**.

Example:

```
interface Vehicle {

   default void stop(){
      System.out.println("Vehicle stopped");
   }
```

```
}
```

Purpose:

- maintain backward compatibility
- extend interfaces without breaking code

Static Methods in Interfaces

Interfaces can define **static utility methods**.

Example:

```
interface MathUtil {

  static int square(int x){
    return x*x;
  }

}
```

Usage:

```
MathUtil.square(5);
```

Functional Interfaces

A functional interface contains **exactly one abstract method**.

Example:

```
@FunctionalInterface
interface Calculator {

  int operate(int a, int b);

}
```

Functional interfaces are heavily used with **Lambda Expressions**.

Example:

```
Calculator add = (a,b) -> a + b;
```

Common built-in functional interfaces exist in **java.util.function**.

Examples:

- Predicate

- Function
- Consumer
- Supplier

Sealed Classes

Sealed classes were introduced in **Java 17**.

They allow a class to **restrict which classes can extend it**.

Example:

```
public sealed class Shape
   permits Circle, Rectangle {
}
```

Allowed subclasses:

```
final class Circle extends Shape {}
final class Rectangle extends Shape {}
```

Why Sealed Classes

Benefits:

- better control over inheritance
- safer API design
- improved pattern matching

Types of Sealed Hierarchy Classes

A permitted subclass must be one of:

Keyword	Meaning
final	cannot be extended
sealed	restricted inheritance
non-sealed	open inheritance

Example:

```
sealed class A permits B {}

final class B extends A {}
```

Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	abstract + concrete	abstract + default + static
Fields	instance variables	constants
Constructors	allowed	not allowed
Multiple inheritance	no	yes

OOP Concept Relationships

Encapsulation → protects data
Inheritance → reuses code
Polymorphism → flexible behavior
Abstraction → hides complexity

Together they form **robust object-oriented design**.

Summary

Principle	Purpose
Encapsulation	Data hiding
Inheritance	Code reuse
Polymorphism	Multiple behavior
Abstraction	Simplify complexity

Normal Interface

A **normal interface** contains **multiple abstract methods** and is the traditional type of interface in Java.

Example:

```
interface Vehicle {

   void start();

   void stop();

}
```

Implementation:

```
class Car implements Vehicle {

   public void start(){
      System.out.println("Car started");
   }

   public void stop(){
      System.out.println("Car stopped");
   }

}
```

Characteristics:

- Can contain multiple abstract methods
- Can contain constants
- Classes implement the interface using implements

Internal Working

When compiled:

Vehicle.java → Vehicle.class

The interface is stored in the **Method Area** by the **Java Virtual Machine**.

Methods inside the interface are implicitly:

public abstract

Variables are implicitly:

public static final

---

2. Functional Interface

A **functional interface** contains **exactly one abstract method**.

Introduced to support **lambda expressions** in **Java 8**.

Example:

```
@FunctionalInterface
interface Calculator {

   int calculate(int a, int b);

}
```

Usage with lambda:

```
Calculator add = (a,b) -> a + b;

System.out.println(add.calculate(5,3));
```

Built-in Functional Interfaces

Java provides several predefined functional interfaces in the **java.util.function**.

Common ones include:

Interface	Purpose
Predicate	boolean condition
Function	input → output
Consumer	consumes input
Supplier	produces output

Example:

```
Predicate<Integer> p = x -> x > 10;
```

---

3. Marker Interface

A **marker interface** is an interface that **contains no methods**.

It simply marks a class and provides special behavior to the JVM or libraries.

Example:

```
interface Marker {
}
```

Real examples include:

- **Serializable**
- **Cloneable**
- **RandomAccess**

Example:

```
class Student implements Serializable {
}
```

Meaning:

The object can be converted into a byte stream.

Internal Working

When the JVM checks serialization:

instanceof Serializable

If the object implements it → JVM allows serialization.

Otherwise → exception occurs.

Example:

NotSerializableException

---

4. Sealed Interface

Introduced in **Java 17**.

A sealed interface restricts **which classes can implement it**.

Example:

```
public sealed interface Shape
   permits Circle, Rectangle {
}
```

Allowed implementations:

```
final class Circle implements Shape {}

final class Rectangle implements Shape {}
```

Why Sealed Interfaces

They help:

- control inheritance
- design safer APIs
- support pattern matching

Example hierarchy:

```
Shape
 ├── Circle
 └── Rectangle
```

No other class can implement Shape.

---

5. Nested Interface

A **nested interface** is an interface declared **inside another class or interface**.

Example:

```
class Outer {
  interface Inner {
    void display();
  }
}
```

Implementation:

```
class Test implements Outer.Inner {
  public void display(){
    System.out.println("Nested interface");
  }
}
```

Usage:

```
Outer.Inner obj = new Test();
```

Why Nested Interfaces

They help:

- logically group interfaces
- improve code organization
- restrict interface scope

Inner Classes in Java

An **Inner Class** is a class defined **inside another class**.

The outer class is called the **enclosing class**, and the class defined inside it is called the **inner class**.

Example:

```
class Outer {

  class Inner {
    void display() {
      System.out.println("Inner class method");
    }
  }

}
```

Inner classes are compiled and executed by the **Java Virtual Machine** like normal classes, but they maintain a **relationship with their outer class**.

Why Inner Classes Exist

Inner classes help:

- logically group related classes
- improve encapsulation
- reduce namespace pollution
- access outer class members easily
- implement event handlers

Types of Inner Classes

Java provides **four main types of inner classes**:

1. Member Inner Class
2. Static Nested Class
3. Local Inner Class
4. Anonymous Inner Class

---

1. Member Inner Class

A **member inner class** is a class declared **inside another class but outside any method**.

Example:

```
class Outer {
  int x = 10;
  class Inner {
    void display() {
      System.out.println("Outer x = " + x);
    }
  }
}
```

Usage:

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();

inner.display();
```

Output:

```
Outer x = 10
```

Internal Working

When compiled:

Outer.java

The compiler generates:

```
Outer.class
Outer$Inner.class
```

The $ symbol represents **nested relationship**.

How JVM Stores It

Memory layout:

```
Method Area
-----------
Outer class metadata
Inner class metadata
```

Heap

Outer object
Inner object

The **Inner object contains a hidden reference to the Outer object**.

Conceptually:

```
Inner {
  Outer outerReference;
}
```

This allows the inner class to access outer class members.

Accessing Outer Class Members

Inner classes can access:

- private fields
- private methods
- protected members

Example:

```
class Outer {
  private int value = 100;
  class Inner {
    void show() {
      System.out.println(value);
    }
  }
}
```

Even though value is private, it is accessible.

Why?

Because both classes belong to the **same compilation unit**.

---

2. Static Nested Class

A **static nested class** is declared using the static keyword.

Example:

```
class Outer {
  static class Inner {
```

```
    void display() {
      System.out.println("Static nested class");
    }
  }
}
```

Usage:

```
Outer.Inner obj = new Outer.Inner();
obj.display();
```

Key Difference

Member Inner Class	**Static Nested Class**
Requires outer object	No outer object needed
Can access instance variables	Cannot access instance variables
Has hidden outer reference	No outer reference

Internal Working

Static nested classes behave like **normal classes scoped inside another class**.

Memory:

```
Method Area
------------
Outer
Outer$Inner

Heap
------------
Inner object
```

No outer object reference is stored.

---

3. Local Inner Class

A **local inner class** is defined **inside a method**.

Example:

```
class Outer {

  void display() {

    class LocalInner {

      void show() {
        System.out.println("Local inner class");
      }

    }

    LocalInner obj = new LocalInner();
    obj.show();

  }

}
```

Usage:

```
new Outer().display();
```

Output:

```
Local inner class
```

Characteristics

Local inner classes:

- exist only inside a method
- cannot be accessed outside
- can access method variables

But the variables must be **final or effectively final**.

Example:

```
void test() {

  int x = 10;
```

```
  class Inner {
    void show() {
      System.out.println(x);
    }
  }

}
```

If x changes later, compilation fails.

Why Final Requirement Exists

Because local variables are stored in **stack memory**, while inner class objects live in **heap memory**.

To avoid inconsistent state, Java requires them to be **effectively final**.

---

4. Anonymous Inner Class

An **anonymous inner class** is a class **without a name**.

It is declared and instantiated **in a single expression**.

Example:

```
abstract class Animal {

  abstract void sound();
}
```

Anonymous implementation:

```
Animal a = new Animal() {

  void sound() {
    System.out.println("Dog barking");
  }

};

a.sound();
```

Internal Compilation

Compiler generates a class like:

Outer$1.class

This represents **the first anonymous class**.

Multiple anonymous classes generate:

Outer$2.class
Outer$3.class

Why Anonymous Classes Are Used

They are commonly used in:

- GUI event handling
- Runnable implementations
- short interface implementations

Example:

```
Thread t = new Thread(new Runnable() {

  public void run() {
    System.out.println("Thread running");
  }

});
```

Anonymous Class vs Lambda

Before **Java 8**, anonymous classes were heavily used.

Example with lambda:

```
Runnable r = () -> System.out.println("Thread running");
```

Lambda expressions replace many anonymous classes.

Inner Class Access Rules

Inner classes can access:

Outer class fields
Outer class methods
Private members

Outer class **cannot directly access inner class members without object creation**.

Example:

```
Outer.Inner obj = new Outer().new Inner();
```

Memory Representation

Example:

```
class Outer {
  int x = 10;

  class Inner {
    void display(){ }
  }
}
```

Memory layout:

```
Stack
---------
reference to Outer
reference to Inner

Heap
---------
Outer object
  x = 10

Inner object
  outerRef → Outer object
```

The **outerRef** pointer allows inner classes to access outer members.

Real World Example

Example GUI listener:

```
button.addActionListener(new ActionListener() {

   public void actionPerformed(ActionEvent e) {
      System.out.println("Button clicked");
   }

});
```

This uses **anonymous inner class**.

Comparison of Inner Class Types

Type	Location	Outer Object Needed
Member Inner Class	inside class	yes
Static Nested Class	inside class with static	no
Local Inner Class	inside method	yes
Anonymous Inner Class	expression	depends

Summary

Inner classes allow **better code organization and encapsulation**.

Main types:

Inner Class Type	Purpose
Member Inner Class	tightly coupled helper class
Static Nested Class	logically grouped class
Local Inner Class	method-specific logic
Anonymous Inner Class	quick one-time implementation

All inner classes are loaded and managed by the **Java Virtual Machine** during execution.

Enums

An **enum (enumeration)** is a special type used to define a **fixed set of constants**.

Enums were introduced in **Java 5** to provide a **type-safe way of representing constants**.

Before enums, developers used:

```
public static final int MONDAY = 1;
public static final int TUESDAY = 2;
```

This approach had problems:

- no type safety
- invalid values possible
- poor readability

Enums solve these issues.

Basic Enum Declaration

Example:

```
enum Day {
   MONDAY,
   TUESDAY,
   WEDNESDAY,
   THURSDAY,
   FRIDAY,
   SATURDAY,
   SUNDAY
}
```

Usage:

```
Day today = Day.MONDAY;

System.out.println(today);
```

Output:

```
MONDAY
```

Why Enums Are Powerful

Enums provide:

- type safety
- readability
- predefined constants
- built-in methods
- ability to define methods and fields

Enums are internally implemented as **special classes handled by the Java Virtual Machine**.

Internal Representation of Enum

When the compiler sees:

```
enum Day { MONDAY, TUESDAY }
```

It internally generates a class similar to:

```
final class Day extends java.lang.Enum<Day> {

   public static final Day MONDAY = new Day("MONDAY",0);
   public static final Day TUESDAY = new Day("TUESDAY",1);

   private Day(String name,int ordinal){
      super(name,ordinal);
   }

}
```

Important facts:

- Every enum **extends java.lang.Enum**
- Enum classes are **implicitly final**
- Enum objects are **created only once**

Enum Object Creation

Example:

```
Day d = Day.MONDAY;
```

Object creation happens **only once during class loading**.

Memory structure:

```
Method Area
-----------
Day class metadata
Enum constants

Heap
-----------
MONDAY object
TUESDAY object
```

Managed by the **Java Virtual Machine**.

Enum Constants Are Objects

Each constant is actually an **object of the enum class**.

Example:

```
Day d = Day.FRIDAY;
```

Here:

FRIDAY → object

Enum Methods

All enums inherit methods from **java.lang.Enum**.

Important methods:

Method	Purpose
values()	returns all enum constants
valueOf()	converts string to enum
ordinal()	returns position
name()	returns constant name

---

values() Method

Example:

```
for(Day d : Day.values()){
   System.out.println(d);
}
```

Output:

```
MONDAY
TUESDAY
WEDNESDAY
...
```

---

valueOf() Method

Example:

```
Day d = Day.valueOf("MONDAY");
System.out.println(d);
```

Output:

```
MONDAY
```

---

ordinal() Method

Returns **index position** of enum constant.

Example:

```
Day d = Day.WEDNESDAY;

System.out.println(d.ordinal());
```

Output:

2

Index starts from **0**.

Enum with Fields

Enums can contain **instance variables**.

Example:

```
enum Size {
   SMALL(28),
   MEDIUM(32),
   LARGE(36);
   int size;
   Size(int s){
      size = s;
   }

}
```

Usage:

```
System.out.println(Size.SMALL.size);
```

Output:

28

Enum with Methods

Enums can define methods like normal classes.

Example:

```
enum Operation {

  ADD,
  SUBTRACT;

  int apply(int a, int b){
    if(this == ADD)
      return a + b;
    else
      return a - b;
  }

}
```

Usage:

```
System.out.println(Operation.ADD.apply(5,3));
```

Output:

```
8
```

Enum Constructors

Enum constructors are always **private**.

Example:

```
enum Status {

  SUCCESS,
  FAILURE;

  Status(){
    System.out.println("Constructor called");
  }

}
```

Enum constructors run **once for each constant** during class loading.

Enum with Abstract Methods

Enums can declare abstract methods.

Example:

```
enum Shape {

  CIRCLE {
    double area(double r){
      return 3.14 * r * r;
    }
  },

  SQUARE {
    double area(double r){
      return r * r;
    }
  };

  abstract double area(double r);

}
```

Usage:

```
System.out.println(Shape.CIRCLE.area(5));
```

Enum and Switch Statement

Enums work perfectly with switch.

Example:

```
Day day = Day.MONDAY;

switch(day){
case MONDAY:
  System.out.println("Start of week");
  break;
case FRIDAY:
  System.out.println("Weekend coming");
  break;

}
```

Switch internally compares **ordinal values**.

Enum Singleton Pattern

Enums provide the **best way to implement Singleton**.

Example:

```
enum Singleton {
  INSTANCE;
  void show(){
    System.out.println("Singleton object");
  }

}
```

Usage:

```
Singleton.INSTANCE.show();
```

Why this works:

- JVM ensures only one instance exists
- thread-safe
- prevents reflection attacks
- prevents serialization issues

Enum and Interfaces

Enums can implement interfaces.

Example:

```
interface Printable{
  void print();
}

enum Message implements Printable {
  HELLO;

  public void print(){
    System.out.println("Hello world");
  }
}
```

Enum Memory Model

Example:

```
enum Color {
  RED, GREEN, BLUE
}
```

Memory layout:

```
Method Area
-----------
Color class metadata
enum constant references


Heap
-----------
RED object
GREEN object
BLUE object
```

Objects are created during **class initialization** by the **Java Virtual Machine**.

Enum Restrictions

Enums cannot:

- extend other classes
- be extended by other classes
- have public constructors

Reason:

Enums already extend **java.lang.Enum**.

Enum Best Use Cases

Enums are best used for:

Scenario	**Example**
Days of week	MONDAY, TUESDAY
Directions	NORTH, SOUTH
Status values	SUCCESS, FAILED
Roles	ADMIN, USER

Enum vs Constant Variables

Feature	Enum	Constants
Type safety	yes	no
Readability	high	low
Methods support	yes	no
Object oriented	yes	no

Real World Example

HTTP status codes:

```
enum HttpStatus {
   OK(200),
   NOT_FOUND(404),
   SERVER_ERROR(500);
   int code;
   HttpStatus(int code){
      this.code = code;
   }
}
```

Usage:

```
System.out.println(HttpStatus.OK.code);
```

Enums are **powerful object-oriented constants**.

Key features:

Feature	Description
Type-safe constants	prevents invalid values
Special class	extends Enum
Objects	each constant is an object
Methods allowed	yes
Constructors allowed	private only

Enums are created and managed internally by the **Java Virtual Machine**.

Exception Handling in Java (Deep Dive)

Exception handling is a mechanism used to **detect and handle runtime problems** during program execution.

Java provides a structured way to handle abnormal situations without crashing the program. All exceptions are managed by the **Java Virtual Machine** during runtime.

Example:

```
int a = 10;
int b = 0;

System.out.println(a / b);
```

Output:

ArithmeticException: / by zero

Without exception handling, the program **terminates immediately**.

What is an Exception?

An **exception** is an **event that disrupts the normal flow of program execution**.

Examples:

- division by zero
- accessing invalid array index
- file not found
- null object reference

Internally, exceptions are **objects created from classes that extend java.lang.Throwable**.

Errors vs Exceptions

Java separates problems into **Errors** and **Exceptions**.

Errors

Errors represent **serious problems that applications should not try to handle**.

Examples:

- **OutOfMemoryError**
- **StackOverflowError**

Example:

```
void recursive(){
   recursive();
}
```

This produces a stack overflow error.

Errors are usually caused by:

- JVM issues
- system failures
- hardware limitations

Applications typically **do not recover from errors**.

Exceptions

Exceptions represent **conditions that applications can handle**.

Example:

```
int arr[] = new int[5];
System.out.println(arr[10]);
```

This produces:

ArrayIndexOutOfBoundsException

Exceptions allow programs to **recover and continue execution**.

Exception Hierarchy

All exception classes follow a hierarchical structure.

```
        Throwable
       /      \
     Error     Exception
                  |
          RuntimeException
```

Root class:

java.lang.Throwable

It has two main subclasses:

1. Error
2. Exception

Checked Exceptions

Checked exceptions are **checked by the compiler during compilation**.

Examples:

- **IOException**
- **SQLException**

Example:

```
FileReader fr = new FileReader("file.txt");
```

Compiler error occurs unless handled.

Solution:

```
try {
  FileReader fr = new FileReader("file.txt");
}
catch(IOException e){
  System.out.println(e);
}
```

Unchecked Exceptions

Unchecked exceptions occur **during runtime** and are **not checked by compiler**.

They extend **RuntimeException**.

Examples:

- **NullPointerException**
- **ArithmeticException**
- **ArrayIndexOutOfBoundsException**

Example:

```
String s = null;
System.out.println(s.length());
```

Output:

```
NullPointerException
```

Exception Propagation

If an exception is not handled in a method, it **propagates up the call stack**.

Example:

```
void m1(){
   m2();
}
void m2(){
   int a = 10/0;
}
```

Execution flow:

m2 → exception occurs
m1 → exception propagates
main → program terminates

This mechanism is implemented by the **Java Virtual Machine**.

---

try-catch Block

The try block contains code that may throw an exception.

The catch block handles it.

Example:

```
try {
  int a = 10 / 0;
}
catch(ArithmeticException e){

  System.out.println("Cannot divide by zero");

}
```

Output:

Cannot divide by zero

Multiple Catch Blocks

Java allows multiple catch blocks.

Example:

```
try{

  int arr[] = new int[5];
  arr[10] = 50;

}
catch(ArrayIndexOutOfBoundsException e){

  System.out.println("Invalid index");
}
catch(Exception e){
  System.out.println("General exception");
}
```

Order rule:

specific → general

---

finally Block

The finally block always executes **whether an exception occurs or not**.

Example:

```
try{
  int a = 10/2;
}
catch(Exception e){
  System.out.println(e);
}
finally{
  System.out.println("Always executed");
}
```

Output:

Always executed

Use case:

close files
close database connections
release resources

---

throw Keyword

The throw keyword **explicitly throws an exception**.

Example:

```
if(age < 18){
   throw new IllegalArgumentException("Not eligible");
}
```

Here an exception object is created and thrown manually.

---

throws Keyword

The throws keyword declares that a method **may throw an exception**.

Example:

```
void readFile() throws IOException {
  FileReader fr = new FileReader("file.txt");
}
```

The calling method must:

- handle the exception
- or declare it further

Custom Exceptions

Developers can create their own exceptions.

Example:

```
class InvalidAgeException extends Exception {

   InvalidAgeException(String msg){
      super(msg);
   }

}
```

Usage:

```
if(age < 18){
  throw new InvalidAgeException("Age must be 18+");
}
```

Custom exceptions help represent **domain-specific errors**.

try-with-resources

Introduced in **Java 7**.

It automatically closes resources.

Example:

```
try(FileReader fr = new FileReader("file.txt")){

  int data = fr.read();
}
catch(IOException e){
  System.out.println(e);
}
```

Resources must implement:

AutoCloseable

Examples:

- FileReader
- BufferedReader
- Database connections

Suppressed Exceptions

Suppressed exceptions occur when **multiple exceptions happen in try-with-resources**.

Example scenario:

1. exception in try
2. exception while closing resource

Example:

```
try(MyResource r = new MyResource()){
  throw new Exception("Main exception");
}
```

If close() also throws exception:

```
Main exception occurs
Close exception suppressed
```

The JVM stores suppressed exceptions internally.

You can retrieve them using:

```
Throwable[] t = e.getSuppressed();
```

Suppressed exceptions were introduced in **Java 7**.

JVM Internal Exception Handling

When an exception occurs:

1. JVM creates exception object
2. Stack trace is recorded
3. Exception table is checked
4. Matching catch block found
5. Control transferred

Bytecode contains an **exception table**.

Example:

from to target type

Meaning:

if exception occurs between from–to
jump to target handler

Stack Trace Mechanism

Example:

```
public static void main(String[] args){
   method1();
}
static void method1(){
   method2();
}

static void method2(){
   int a = 10/0;
}
```

Stack trace:

```
Exception in thread "main" java.lang.ArithmeticException
   at method2()
   at method1()
   at main()
```

This shows **call stack order**.

Best Practices for Exception Handling

1. Catch specific exceptions

2. Avoid catching generic Exception unnecessarily
3. Use finally or try-with-resources
4. Create meaningful custom exceptions
5. Do not ignore exceptions silently

Concept	Description
Error	serious JVM problem
Exception	recoverable problem
Checked Exception	compile-time checking
Unchecked Exception	runtime error
try-catch	handle exceptions
finally	always executes
throw	manually throw exception
throws	declare exception
Custom Exception	user-defined error
try-with-resources	automatic resource closing
Suppressed Exception	hidden secondary exception

All exception handling mechanisms are implemented and managed by the **Java Virtual Machine**.